ORBITGUARD

Real-Time Space Situational Awareness: 3D Orbit, Debris Conjunction & Collision-Avoidance Console

Course: CS3002-1 Computer Graphics and Visualization (IPCC), NMAMIT Nitte. Graphics Mini Project **Stack:** C++17 · OpenGL (fixed-function + optional GLSL) · freeGLUT · GLM · Dear ImGui (GLUT backend) · stb_image · nlohmann/json · CMake

1. Abstract

ORBITGUARD is a real-time 3D Space Situational Awareness (SSA) console, built in C++ with OpenGL/GLUT, that visualizes Earth-orbiting satellites and thousands of debris objects, screens them for close approaches, and plans and visualizes collision-avoidance maneuvers. It is modeled on the workflow of a satellite operator's mission-control screen: load a catalog, watch the orbital environment evolve, receive conjunction alerts, inspect a threat, and commit a burn.

Satellites are loaded from real Two-Line Element (TLE) data (a bundled offline snapshot) and propagated using Keplerian orbital mechanics with J2 perturbation. The orbital-element-to-position pipeline is built entirely from composed 3D rotation matrices in homogeneous coordinates, so the physics is literally a graphics transformation chain. A two-phase conjunction engine (uniform spatial-grid broad phase + time-of-closest-approach narrow phase) screens thousands of objects every frame and assigns risk tiers. When a threat is detected, an avoidance planner searches for the minimum along-track Δv burn that restores a safe miss distance, and renders the old and new orbits side by side.

Two features set the project apart from typical orbit visualizers:

1. **Kessler Cascade Mode.** A collision spawns a fractal-generated debris cloud that propagates and can trigger chain collisions, with a live cascade counter.
2. **Algorithm Lab.** The same scene can be rendered with different textbook algorithms (Z-buffer vs. painter's list-priority, flat vs. Gouraud shading, aliased vs. antialiased lines, RGB/HSV/YIQ color modes) and compared live. The syllabus becomes a visible, demonstrable feature instead of hidden plumbing.

The system prioritizes clarity and performance over production-grade astrodynamics. It is an educational, computationally efficient platform showing how core computer graphics principles apply to a real aerospace problem.

> **Honesty note for viva/interviews:** propagation is simplified Kepler + J2 (not full SGP4), and collision probability is an approximate Gaussian estimate with assumed covariance. Say this up front; it reads as engineering maturity, not weakness.

2. Recruiter Pitch (memorize this)

30-second pitch: "I built ORBITGUARD, a real-time C++/OpenGL space-traffic console. It propagates real satellite catalog data, screens thousands of debris objects for collisions every frame using a spatial-hash broad phase, and automatically plans the minimum-fuel avoidance burn when a threat appears. It also simulates Kessler cascades, the chain-reaction debris problem that SpaceX and ISRO actually worry about. I implemented the classic graphics algorithms myself, like midpoint rasterization, Cohen-Sutherland clipping and color-ID picking, and built a mode where you can compare them live."

Resume bullets (fill in measured numbers only after benchmarking):

- Built a real-time 3D space-traffic visualization system in C++/OpenGL rendering **N,000+** orbital objects at **XX FPS**, propagating real TLE catalog data via Keplerian mechanics with J2 perturbation.
- Designed a two-phase conjunction-screening engine (uniform spatial grid + TCA search) cutting pairwise checks from $O(n^2)$ to near-linear, with HSV-graded risk tiers and approximate collision probability.
- Implemented an automated collision-avoidance planner that searches minimum-Δv along-track burns and visualizes pre/post-maneuver orbits with Bezier transfer arcs.
- Hand-implemented raster algorithms (midpoint line/circle, scan-line fill, Cohen-Sutherland & Sutherland-Hodgman clipping) for a custom mission-control HUD, plus GPU color-ID object picking.

3. Feature Set

3.1 Core (MVP, must ship)

#	Feature	Description
F1	**Earth & Space Scene**	Textured, lit Earth (UV-sphere mesh), day/night terminator from a directional "sun" light, procedural starfield, slow Earth rotation synced to sim time.
F2	**TLE Catalog Loader**	Parse a bundled `data/catalog.tle` (snapshot of active sats + debris). Extract a, e, i, Ω, ω, M_0, epoch. Fall back to a JSON scenario file.
F3	**Orbital Propagator**	Solve Kepler's equation (Newton-Raphson), perifocal → ECI via `Rz(Ω)·Rx(i)·Rz(ω)`, J2 secular drift of Ω and ω.
F4	**Satellite Models**	Polygon-mesh satellites (bus cube + solar-panel quads + antenna), oriented along velocity. Debris as point sprites (far) / fractal meshes (near, LOD).
F5	**Orbit Trails & Paths**	Full predicted orbit as sampled closed curve; fading history trail; ground track.
F6	**Time Control**	Pause, play, time-warp (1× to 10,000×), step, scrub. UTC sim clock in HUD.
F7	**Conjunction Screening**	Broad phase: uniform 3D grid (cell ≈ 50 km) rebuilt each tick. Narrow phase: relative-motion sampling over a look-ahead window, refined by golden-section search for TCA and miss distance.
F8	**Risk Tiers & Alerts**	GREEN / YELLOW / AMBER / RED by miss distance (configurable demo thresholds). Threat lines drawn between pairs, color interpolated in HSV. Alert feed panel with timestamps.
F9	**Avoidance Planner**	For a selected conjunction: search prograde/retrograde Δv (vis-viva → new a), re-run TCA, pick minimum Δv meeting safe distance. Show old orbit (dashed, stippled) vs new orbit (solid) + Bezier transfer arc. "Commit Burn" button.
F10	**Object Picking & Inspector**	Click any object → GPU color-ID picking. Inspector shows NORAD ID, name, altitude, velocity, period, orbital elements, nearest threat.
F11	**Cameras**	Arcball orbit camera, follow-satellite camera, free-fly; perspective ⇄ orthographic toggle.
F12	**Mission-Control HUD**	Custom-rendered radar scope + 2D ground-track minimap + status bar (FPS, object count, active alerts, sim time).

3.2 Differentiators (high pitch value)

#	Feature	Description
D1	**Kessler Cascade Mode**	A RED conjunction left unresolved (or a user-triggered "ASAT test") becomes a collision: spawns 200 to 1,000 fragments with velocity dispersion; fragments are screened too, so chain collisions emerge. Cascade counter + debris-population graph.
D2	**Algorithm Lab**	Side panel that switches render algorithms live: Z-buffer ⇄ painter's (list priority); flat ⇄ Gouraud shading; antialiasing on/off; color mode RGB / HSV risk heatmap / YIQ "night-ops" monochrome / CMY "print" preview; wireframe with hidden-line removal. Shows a one-line explanation of the active algorithm.
D3	**Quad-View Layout**	CAD-style 4-viewport mode: perspective + top (ortho) + side (ortho) + ground-track. Demonstrates multiple projections and window-to-viewport mapping at once.
D4	**Cinematic Mission Tour**	Camera flies a B-spline path through scenario keyframes (Earth wide → satellite close-up → conjunction → burn). One key → demo video for LinkedIn/GitHub.
D5	**Conjunction Report Export**	Export current alerts to `reports/conjunctions_<timestamp>.csv` (pair, TCA, miss distance, Pc, recommended Δv). This mirrors real operator output.
D6	**Ground Stations & Visibility**	Stations (e.g., Bengaluru ISTRAC, Sriharikota, Svalbard) with line-of-sight cones; links light up when a satellite is above the elevation mask; coverage footprints on the minimap.
D7	**Scenario System**	JSON scenarios (`scenarios/*.json`) for reproducible demos: "Near-miss in LEO", "Kessler event", "Constellation shell". Deterministic seeded RNG for replay.

3.3 Stretch (only if time remains)

- Per-pixel Phong via GLSL (compare against fixed-function Gouraud in Algorithm Lab).
- Software-raster "Occlusion Inspector": small CPU-rendered panel showing Warnock area-subdivision quads and scan-line spans.
- Live TLE refresh from CelesTrak (keep offline snapshot as default; demo must not depend on internet).

---

4. Syllabus Coverage Map (CS3002-1)

This table is your viva weapon. Every unit and every suggested experiment maps to visible code.

Syllabus Topic	Where it lives in ORBITGUARD	File
Raster vs random scan, video	Explained in README; HUD drawn pixel-by-pixel to raster buffer	`docs/`, `hud/`

controller		
Midpoint line algorithm	Radar sweep line, HUD borders, threat vectors on minimap	`raster/Midpoint.cpp`
Midpoint circle algorithm	Radar range rings, station range circles	`raster/Midpoint.cpp`
Scan-line polygon fill	Filled coverage footprints & HUD warning panels	`raster/ScanlineFill.cpp`
Cohen-Sutherland line clipping	Ground-track segments clipped to minimap window	`raster/Clipping.cpp`
Sutherland-Hodgman polygon clipping	Station coverage polygons clipped to minimap rect	`raster/Clipping.cpp`
Antialiasing	`GL_LINE_SMOOTH` /MSAA toggle + Xiaolin Wu lines in HUD	`raster/Antialias.cpp`
2D transforms, homogeneous coords, composition	HUD radar blips (rotate/scale/translate), minimap pan/zoom	`math/Transform2D.cpp`
Window-to-viewport transformation	km → radar pixels; lat/lon → minimap pixels; quad-view	`hud/Viewport.cpp`
3D transforms (matrix form, composition)	Orbital element → ECI chain `Rz(Ω)Rx(i)Rz(ω)` ; satellite attitude; camera	`orbit/Propagator.cpp` , `math/Transform3D.cpp`
Perspective & parallel projection	Main view perspective, ortho toggle, quad-view	`render/Camera.cpp`
Color models RGB, CMY, YIQ, HSV	Hand-written converters; HSV risk gradient; YIQ night-ops; CMY print preview	`color/ColorModels.cpp`
Polygon meshes	Earth UV-sphere, satellite models, station dishes	`geom/Mesh.cpp`
Bezier curves	Maneuver transfer arcs	`geom/Curves.cpp`
B-spline curves	Cinematic camera tour path	`geom/Curves.cpp`
Fractal geometry	Debris meshes via midpoint-displacement subdivision; Kessler fragment shapes	`geom/Fractal.cpp`
Illumination models	Ambient + diffuse + specular sun light; Earth day/night	`render/Lighting.cpp`
Shading models	Flat ⇄ Gouraud toggle (Phong stretch)	`render/Lighting.cpp`
Coherence	Frame-to-frame coherence: grid reuses previous-frame cells; only moved objects rehashed	`ssa/SpatialGrid.cpp`
Z-buffer	Default depth handling	`render/Renderer.cpp`
List priority (painter's)	Depth-sorted rendering for translucent shells/footprints; Algorithm Lab toggle	`render/Renderer.cpp`
Scan-line / area subdivision	Stretch: Occlusion Inspector	`lab/`
OpenGL primitives, attributes, control functions	Throughout; GLUT callbacks for display/idle/reshape/mouse/keyboard	`app/`
Expt: 2D & 3D transformations	HUD + orbital chain	✔
Expt: Clipping	Minimap	✔
Expt: 3D viewing	Cameras + projections + quad-view	✔
Expt: Hidden line removal	Wireframe + hidden-line mode (polygon offset)	✔
Expt: Fractal generation	Debris meshes + Kessler cloud	✔
Expt: Line/circle using primitives only	Midpoint algorithms plotting `GL_POINTS`	✔

> **Check with your faculty:** whether Dear ImGui is acceptable for control panels. If not, set `ORBITGUARD_USE_IMGUI=OFF` and the panels fall back to GLUT bitmap text + custom-drawn buttons (the HUD is custom-drawn regardless).

5. Physics & Math Specification

Constants: $\mu = 398600.4418$ km³/s², $R_\oplus = 6378.137$ km, $J2 = 1.08263\times10^{-3}$, $\omega_\oplus = 7.2921159\times10^{-5}$ rad/s. **Render scale:** 1 GL unit = $R_\oplus$ (Earth radius = 1.0). Keep physics in km/double; convert to float only at render.

Propagation per object per tick:

1. $n = \sqrt{\mu/a^3}$; $M = M_0 + n\cdot\Delta t$
2. Solve $E - e\cdot\sin E = M$ (Newton-Raphson, ≤ 10 iterations, tol 1e-10)
3. $\nu = 2\cdot\text{atan2}(\sqrt{1+e}\cdot\sin(E/2), \sqrt{1-e}\cdot\cos(E/2))$; $r = a(1 - e\cdot\cos E)$
4. Perifocal position $(r\cdot\cos\nu, r\cdot\sin\nu, 0)$ → ECI via **$R = R_z(\Omega)\cdot R_x(i)\cdot R_z(\omega)$** (built with the project's own 4×4 homogeneous matrix code)
5. J2 secular rates: $\Omega = -1.5\cdot n\cdot J2\cdot(R_\oplus/p)^2\cdot\cos i$; $\omega = 0.75\cdot n\cdot J2\cdot(R_\oplus/p)^2\cdot(5\cos^2 i - 1)$, with $p = a(1-e^2)$

6. ECI → ECEF for ground track by rotating by Earth rotation angle (GMST).

Conjunction screening (every K ticks, K configurable):

- Broad phase: hash each object into a uniform grid; test only same/neighbor cells (27-cell neighborhood).
- Narrow phase: for candidate pairs, sample relative distance over look-ahead window T (default 30 min sim time); refine minimum with golden-section search → **TCA, miss distance d**.
- Approximate collision probability: 2D Gaussian in encounter plane with assumed combined σ (config) and hard-body radius; label clearly as "approx. Pc".
- Risk tiers (demo-scaled, configurable in `config.json`): RED d < 1 km, AMBER < 5 km, YELLOW < 25 km.

Avoidance planner:

- Candidate burns: along-track Δv ∈ ±{0.01 … 2.0} m/s (log-spaced).
- New a from vis-viva: $v^2 = \mu(2/r - 1/a)$. Re-propagate, recompute TCA vs threat and vs all other neighbors (do not create a new conjunction).
- Choose minimum |Δv| giving d ≥ safe threshold. Display Δv, new miss distance, and "fuel cost" (Tsiolkovsky with assumed Isp/mass).

Kessler fragments: n_frag ∝ impact energy (simplified); each fragment's velocity = parent v + random dispersion (seeded); fragments inherit screening.

---

6. Architecture

```
orbitguard/
├── CMakeLists.txt
├── README.md                # screenshots, GIF, build steps, syllabus map
├── config.json              # thresholds, warp limits, grid size, colors
├── data/catalog.tle         # offline TLE snapshot
├── assets/textures/earth_day.jpg, earth_night.jpg, stars.png
├── scenarios/               # leo_near_miss.json, kessler_event.json, constellation.json
├── reports/                 # CSV exports (gitignored)
├── external/                # glm, imgui, stb_image, nlohmann_json
└── src/
    ├── main.cpp
    ├── app/        App.cpp, Input.cpp, SimClock.cpp           # GLUT callbacks, state machine
    ├── math/       Mat4.cpp, Transform2D.cpp, Transform3D.cpp   # own homogeneous matrix code
    ├── orbit/      TLEParser.cpp, OrbitalElements.cpp, Propagator.cpp
    ├── ssa/        SpatialGrid.cpp, Conjunction.cpp, AvoidancePlanner.cpp, Kessler.cpp
    ├── geom/       Mesh.cpp, Curves.cpp (Bezier, B-spline), Fractal.cpp
    ├── render/     Renderer.cpp, Camera.cpp, Lighting.cpp, Picking.cpp, Starfield.cpp
    ├── raster/     Midpoint.cpp, ScanlineFill.cpp, Clipping.cpp, Antialias.cpp
    ├── color/      ColorModels.cpp
    ├── hud/        Radar.cpp, GroundTrackMap.cpp, Viewport.cpp, AlertFeed.cpp, StatusBar.cpp
    ├── ui/         Panels.cpp (ImGui or fallback)
    ├── lab/        AlgorithmLab.cpp
    └── io/         Scenario.cpp, ReportExporter.cpp, Config.cpp
tests/              test_kepler.cpp, test_clipping.cpp, test_color.cpp, test_grid.cpp
```

Design rules:

- Simulation (`orbit/`, `ssa/`) has **zero OpenGL dependencies**, so it's unit-testable and reusable.
- Fixed timestep simulation (e.g., 50 Hz) decoupled from render rate; interpolate for rendering.
- Struct-of-arrays for objects (positions, velocities, elements) for cache-friendly propagation of thousands of objects.
- All tunables in `config.json`; no magic numbers in code.

---

7. Controls

Input	Action
Left-drag / scroll	Arcball rotate / zoom
Left-click	Pick object (inspector)
`F`	Follow selected object
`P`	Perspective ⇄ orthographic
`Q`	Quad-view layout
`Space` / `+` / `-`	Pause / warp up / warp down
`A`	Plan avoidance for selected conjunction · `Enter` commit burn

`K`	Trigger Kessler event on selected object
`L`	Toggle Algorithm Lab panel
`T`	Start cinematic tour
`E`	Export conjunction report
`1`–`4`	Color mode: RGB / HSV heatmap / YIQ night-ops / CMY preview
`W`	Wireframe + hidden-line removal
`Esc`	Quit

8. Build Plan for Claude Code (phased)

Give Claude Code one phase at a time. Don't move on until the acceptance check passes.

Phase	Scope	Acceptance check
0. Scaffold	CMake, freeGLUT window, deps in `external/`, config loader, empty render loop, FPS counter	Window opens at 60 FPS on Windows and Linux
1. Math core	Own `Mat4`, 2D/3D transform builders, unit tests	`tests/` pass; compose rotations matches GLM to 1e-6
2. Earth & camera	UV-sphere mesh, texture, lighting, starfield, arcball, perspective/ortho	Lit rotating textured Earth, smooth camera
3. Orbits	TLE parser, Kepler solver, propagator + J2, sat meshes, orbit paths, trails, sim clock + warp	ISS-like orbit period ≈ 92 min; loads full catalog snapshot
4. Picking & inspector	Color-ID picking, inspector panel	Click selects the correct object 100% in test scene
5. Raster HUD	Midpoint line/circle, scan-line fill, both clippers, Wu AA, window-to-viewport; radar + minimap + status bar	Ground track wraps cleanly; clipping unit tests pass
6. Conjunction engine	Spatial grid, TCA search, risk tiers, threat lines (HSV), alert feed	`leo_near_miss.json` raises RED alert at expected TCA
7. Avoidance planner	Δv search, dashed old orbit, solid new orbit, Bezier arc, commit burn	Post-burn miss distance ≥ safe threshold, no new conjunction
8. Kessler mode	Fractal debris meshes, fragment spawn, chain screening, cascade counter/graph	`kessler_event.json` produces a reproducible cascade (seeded)
9. Algorithm Lab & views	Z-buffer⇄painter's, flat⇄Gouraud, AA toggle, color modes, hidden-line wireframe, quad-view	Every toggle visibly changes output
10. Polish & ship	Ground stations, B-spline tour, CSV export, README with GIF + syllabus map, benchmark numbers	Clean build from fresh clone; recorded demo

Performance targets (measure, then put real numbers on your resume): ≥ 5,000 objects at ≥ 60 FPS on a mid-range laptop GPU; screening pass < 5 ms.

9. Kickoff Prompt for Claude Code

Paste this with the spec file in the repo root:

```
Read ORBITGUARD_SPEC.md fully. We are building it phase by phase per Section 8.
Start with Phase 0 only. Constraints:
- C++17, CMake, freeGLUT + legacy OpenGL (GLSL only where the spec says optional).
- Must build on Windows (MSVC or MinGW) and Linux. Vendor header-only deps into external/.
- Simulation code in src/orbit and src/ssa must not include any OpenGL headers.
- Implement the raster algorithms (midpoint, scan-line fill, Cohen-Sutherland,
  Sutherland-Hodgman, color-model conversions, 4x4 matrices) by hand. Do not
  replace them with library calls. They are graded syllabus items.
- After each phase: build, run the acceptance check from Section 8, summarize
  what changed, and stop for my review before the next phase.
```

10. Deliverables Checklist

- [ ] Source on GitHub with clean commit history (one commit series per phase)
- [ ] README: hero GIF, feature list, syllabus map (Section 4), build steps, controls
- [ ] 90-second demo video using the cinematic tour (LinkedIn + resume link)

- [ ] Mini-project report: abstract, architecture diagram, algorithm explanations with screenshots from Algorithm Lab, results/benchmarks, limitations, future work (SGP4, real covariance data, GPU compute screening)